The binding models populated by the Razor Pages infrastructure are passed to page handlers when they execute. Once the page handler has run, you're all set up to use the output models in ASP.NET Core's implementation of Model-View-Controller (MVC): the view models and API models. These are used to generate a response to the user's request. We'll cover them in chapters 19 and 20.

Before we go any further, let's recap the MVC design pattern and how binding models fit into ASP.NET Core.

16.1 Understanding the models in Razor Pages and MVC

In this section I describe how binding models fit into the MVC design pattern we covered in chapter 13. I describe the difference between binding models and the other "model" concepts in the MVC pattern and how they're each used in ASP.NET Core.

MVC is all about the separation of concerns. The premise is that isolating each aspect of your application to focus on a single responsibility reduces the interdependencies in your system. This separation makes it easier to make changes without affecting other parts of your application.

The classic MVC design pattern has three independent components:

- *Model*—The data to display and the methods for updating this data
- *View*—Displays a representation of data that makes up the model
- *Controller*—Calls methods on the model and selects a view

In this representation, there's only one model, the application model, which represents all the business logic for the application as well as how to update